Marcos Antônio Ferreira de Araújo

Matheus de Castro Viana

Relatório Técnico DevSecOps Avançado no ERP Escolar

Mogi das Cruzes

2026

1. Introdução e Contextualização

O School ERP é um sistema de gestão escolar que trata dados pessoais de funcionários, responsáveis e alunos. Por isso, ele precisa de autenticação robusta, criptografia de dados em repouso e conformidade com a LGPD (Lei nº 13.709/2018), que exige em seu art. 46 a adoção de medidas de segurança técnicas e administrativas aptas a proteger os dados pessoais.

Tradicionalmente, a segurança era verificada ao final do desenvolvimento, quando corrigir uma falha é mais caro e mais arriscado. O SecDevOps (ou DevSecOps) propõe o caminho inverso: incorporar a segurança em todo o ciclo de desenvolvimento (shift-left) e automatizá-la dentro da integração contínua. Assim, cada alteração de código passa pelas mesmas verificações, sem depender da memória ou da disponibilidade de uma pessoa.

A automação traz três ganhos principais. O primeiro é a repetibilidade: a mesma verificação roda em todo push e pull request. O segundo é o feedback rápido: o desenvolvedor descobre o problema minutos depois de introduzi-lo. O terceiro é a rastreabilidade: cada execução fica registrada com data, commit, autor e resultado, o que serve de evidência para auditoria. No projeto, três riscos recorrentes motivaram a escolha das verificações:

- Segredos expostos (chaves e senhas no histórico do Git), ligados à categoria Cryptographic Failures (A02:2021) do OWASP Top 10;
- Dependências vulneráveis de terceiros, ligadas à categoria Vulnerable and Outdated Components (A06:2021);
- Falhas no próprio código, como configuração insegura e uso inadequado de criptografia, ligadas a Security Misconfiguration (A05:2021) e Injection (A03:2021).

2. Descrição das Ferramentas Adotadas

A esteira está em .github/workflows/devsecops-pipeline.yml e é executada a cada push e a cada pull_request. Ela tem três jobs independentes, um por vertente de segurança, executados em paralelo em ubuntu-24.04. As permissões do workflow seguem o princípio do menor privilégio (contents: read), e cada ferramenta roda em um contêiner Docker, o que isola a execução e padroniza o ambiente.

Separar as vertentes em jobs distintos faz com que uma falha em uma delas não esconda as outras e permite marcar cada job como verificação obrigatória na proteção da branch.

2.1 Varredura de segredos TruffleHog

O job faz o checkout com fetch-depth: 0 (histórico completo) e executa o trufflehog git file:///repo --fail --no-update. Analisa todo o histórico do Git, não apenas a versão atual dos arquivos. Isso importa porque um segredo commitado e depois apagado continua acessível no histórico. Possui centenas de detectores de credenciais e consegue verificar se a credencial encontrada ainda é válida, o que reduz falsos positivos. A opção --fail faz o processo terminar com código de saída diferente de zero (183) quando encontra segredos, o que bloqueia o build. É de código aberto e roda como contêiner, sem depender de serviço pago.

2.2 Análise de dependências (SCA) Trivy

O job executa trivy fs --scanners vuln --severity HIGH,CRITICAL --ignore-unfixed --exit-code. Ele lê o requirements.txt e compara cada biblioteca e sua versão com bases públicas de vulnerabilidades (como a GitHub Advisory Database e a NVD). Cobre as bibliotecas de terceiros, que concentram parte relevante do risco do sistema, inclusive as usadas em segurança (cryptography, argon2-cffi, django-allauth, pillow). O filtro HIGH,CRITICAL e a opção --exit-code 1 implementam diretamente a política de bloqueio pedida.

A opção --ignore-unfixed ignora vulnerabilidades que ainda não têm versão corrigida, mantendo os alertas acionáveis: tudo o que bloqueia pode ser resolvido atualizando uma dependência. O projeto já usa o Trivy na esteira anterior (security-pipeline.yml) para gerar o SBOM em CycloneDX. Usar a mesma ferramenta mantém a cadeia de suprimentos coerente.

2.3 Análise estática (SAST) Semgrep

O job executa semgrep scan --config p/python --config p/django --severity ERROR --error --metrics off.As regras p/python e p/django do registro oficial do Semgrep cobrem padrões inseguros típicos de aplicações Django: configuração insegura, injeção, uso de criptografia fraca e tratamento inadequado de dados sensíveis. A severidade ERROR do Semgrep corresponde às falhas de severidade alta/crítica. A opção --error faz o processo terminar com código de saída 1 quando há qualquer achado nesse nível, bloqueando o build. É rápido, não exige configuração adicional nas configurações do repositório e permite regras personalizadas. O repositório é público, o que permitiria usar também o CodeQL como alternativa. A escolha pelo Semgrep se deu pelo controle direto do código de saída e pelas regras específicas para Django.

2.4 Política de bloqueio

Vertente	**Ferramenta**	**Critério de falha**	**Código de saída**
Segredos	TruffleHog	Qualquer segredo no histórico do Git	183 (--fail)
Dependências (SCA)	Trivy	Vulnerabilidade HIGH ou CRITICAL com correção disponível	1 (--exit-code 1)
Análise estática (SAST)	Semgrep	Achado de severidade ERROR	1 (--error)

Quando qualquer job falha, o GitHub marca a verificação como reprovada. Com as verificações definidas como obrigatórias na proteção da branch, o merge é impedido.

2.5 Verificações complementares

Além dos três jobs exigidos, o repositório mantém outros dois workflows que ampliam a cobertura:

- Unit Tests (testes.yml): executa a suíte de testes unitários em PostgreSQL 18, com relatório de cobertura (82%).
- Security Pipeline: análise do Bearer, geração do SBOM com Trivy e teste dinâmico (DAST) com OWASP ZAP.

3. Integração com o "Requisitos 1.pdf"

O Documento de Requisitos do projeto define critérios de verificação como "inspeção das configurações", "inspeção do código ou da biblioteca" e "execução dos testes de segurança correspondentes". Verificações manuais e pontuais perdem validade a cada novo commit. A esteira transforma esses critérios em verificações contínuas e registradas.

Requisito	O que o documento exige	Como a esteira apoia	Ferramenta
3.6	Chaves criptográficas protegidas, sem estar no código-fonte nem em repositórios públicos	Varre todo o histórico e bloqueia o build se houver credencial exposta	TruffleHog
3.7	Estratégia de criptografia com ciclo de vida, rotação e recuperação de chaves	A detecção de uma exposição aciona a rotação de chaves	TruffleHog
3.5 e 3.8	Algoritmo adequado (como AES em modo seguro) e bibliotecas e configurações compatíveis com os dados tratados	O SCA garante que as bibliotecas de criptografia e hash não tenham CVEs graves conhecidos; o SAST aponta uso de algoritmos fracos	Trivy e Semgrep
1.12	Configurações coerentes de hash, 2FA, sessão e proteção contra força bruta, verificadas por inspeção e execução de testes	O SAST verifica a configuração do Django a cada commit; os testes unitários de segurança rodam em todo push	Semgrep e Unit Tests
2.2	Token de recuperação gerado por mecanismo criptograficamente seguro	O SAST sinaliza geradores aleatórios inseguros; os testes validam o formato do token	Semgrep e Unit Tests
Seção 8 (LGPD)	Proteção dos dados pessoais efetivamente coletados	Materializa as medidas técnicas de segurança do art. 46 da LGPD e mantém a trilha de evidências	Todas

Auditoria e conformidade. Cada execução da esteira fica registrada no GitHub Actions com o commit, o autor, a data e o resultado de cada ferramenta. Esse histórico funciona como evidência de verificação contínua, e atende os itens 6.7 (identificação de ameaças e vulnerabilidades), 6.9 (testes de segurança realizados) e 6.10 (resultados dos testes documentados) do checklist de entrega final.

Proteção contínua. Como a esteira roda em todo push e pull_request, uma regressão (por exemplo, uma chave commitada por engano ou uma dependência com CVE) é barrada antes de chegar à branch principal, e não descoberta numa auditoria posterior.

Um episódio real do projeto ilustra o requisito 3.7: a chave de desenvolvimento gerada pelo Django estava no settings.py do commit inicial e foi substituída por variável de ambiente em 27/09/2026 (commit c065cf6, "Fix: Chave exposta. Gerada nova chave e adicionada ao .env"). A esteira tem o papel de impedir que esse tipo de falha se repita sem ser notado.

4. Evidências de Funcionamento

Figura 1: Jobs TruffleHog, Trivy - Dependency vulnerabilities (SCA) e Semgrep - Static analysis (SAST) em verde e vermelho

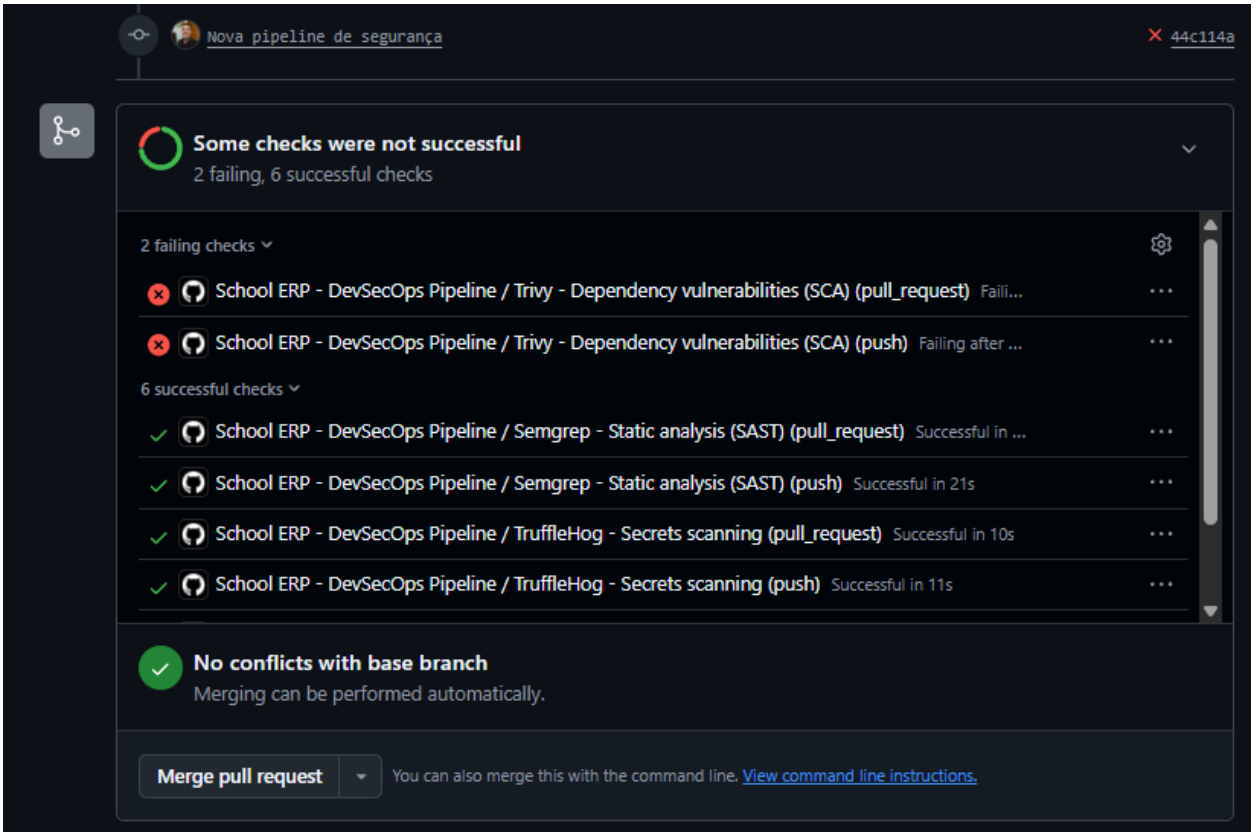


Figura 2: PR com o resumo 8 successful checks

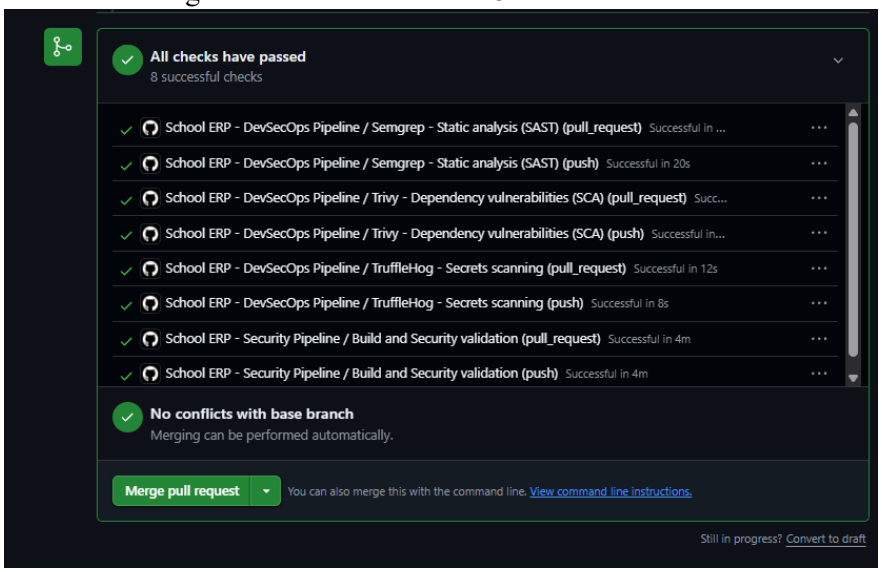
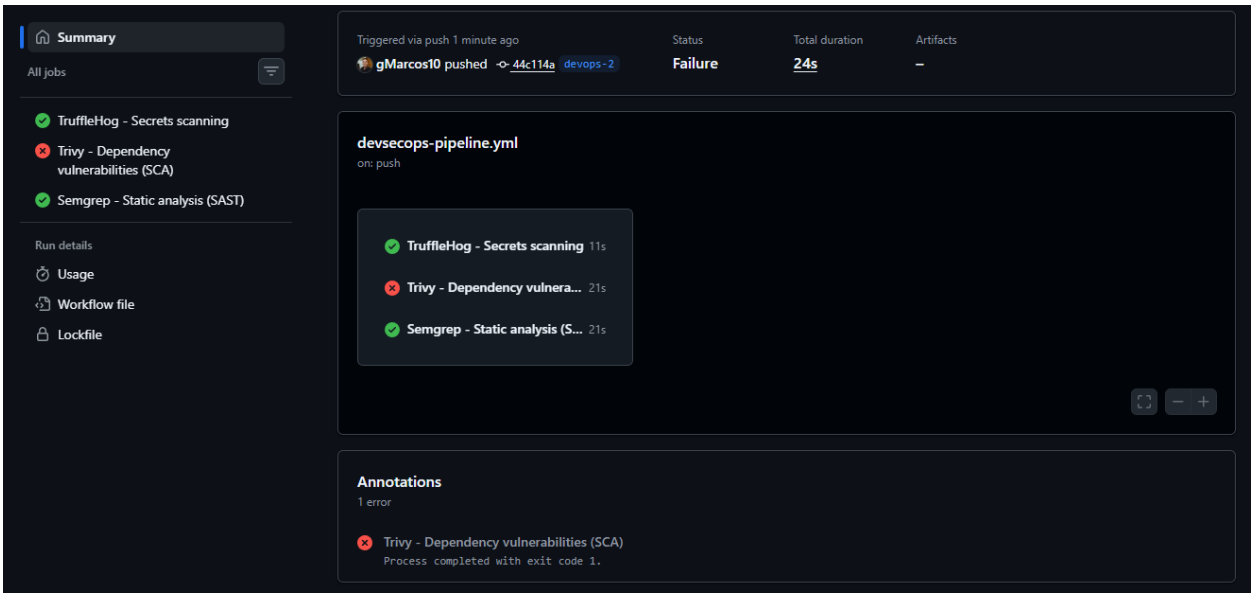


Figura 3: Execução do Trivy com falha



4.1 Alertas encontrados e mitigação

Todas as bibliotecas afetadas já tinham versão corrigida. A mitigação consistiu em atualizar o requirements.txt (commit afe2703, Atualização de dependências):

Biblioteca	Versão vulnerável	Versão corrigida	Mitigação
pillow	12.3.0	13.0.0	Atualização de versão
urllib3	2.7.0	2.8.0	Atualização de versão
certifi	2026.7.22	2026.10.18	Atualização de versão

Após a atualização, a esteira foi executada novamente e o Trivy deixou de apontar vulnerabilidades HIGH/CRITICAL. A compatibilidade das novas versões foi validada pela suíte de testes unitários (workflow Unit Tests), executada no mesmo commit.

O Dependabot e o CodeQL não foram utilizados nesta entrega, portanto não há alertas desses recursos no painel de segurança. As evidências de alertas e de mitigação são as do Trivy.

4.2 Testes unitários e cobertura

O workflow Unit Tests executa a suíte completa a cada alteração e publica o relatório de cobertura. O resultado foi de 82% de cobertura (1.317 instruções, 233 não cobertas), com 100% nos

módulos de criptografia (config/criptografia.py, config/campos.py) e de permissões (usuarios/permissoes.py).

Referências

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF, 2018.

OWASP FOUNDATION. OWASP Top 10:2021. 2021. Disponível em: https://owasp.org/Top10/. Acesso em: 11 nov. 2026.

AQUA SECURITY. Trivy Documentation. Disponível em: https://trivy.dev/. Acesso em: 11 nov. 2026.

SEMGREP. Semgrep Documentation. Disponível em: https://semgrep.dev/docs/. Acesso em: 11 nov. 2026.

TRUFFLE SECURITY. TruffleHog Documentation. Disponível em: https://github.com/trufflesecurity/trufflehog. Acesso em: 11 nov. 2026.